



Machine learning

Regression

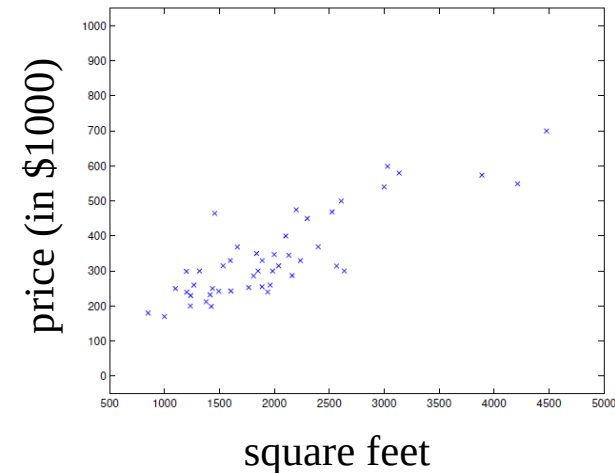
(adapted from Stanford ML regression notes)

Mohammad-Reza A. Dehaqani
dehaqani@ut.ac.ir

Prices of houses



Living area (feet ²)	#bedrooms	Price (1000\$s)
2104	3	400
1600	3	330
2400	3	369
1416	2	232
3000	4	540
⋮	⋮	⋮



- Given data like this, how can we **learn** to **predict** the prices of other houses, **as a function** of the size of their living areas?
- A pair $(x^{(i)}, y^{(i)})$ is called a **training example**, and the dataset that we'll be using to learn—a list of n training examples $\{(x^{(i)}, y^{(i)}); i=1, \dots, n\}$ —is called a **training set**.
- We used superscript “(i)” in the notation for regression to denote an index into the data set. In other section, we usually use **subscript**.

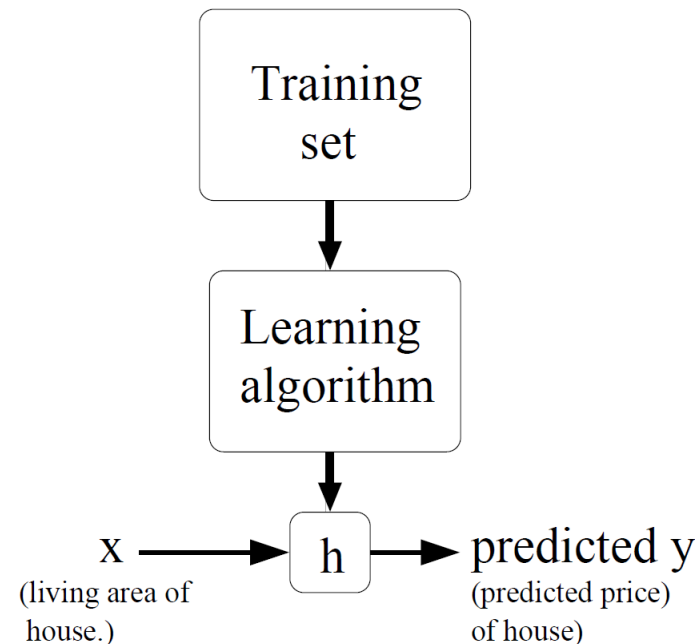
Linear Regression

- We approximate y as **a linear function** of x :

$$h_{\theta}(x) = \theta_0 + \theta_1 x_1 + \theta_2 x_2$$

- To perform **supervised learning**, we must decide how we're going to **represent** functions/hypotheses h in a computer.
- The θ_i 's are the **parameters** (also called weights) **parameterizing** the space of linear functions mapping from X to Y
- Letting $x_0 = 1$ (this is the **intercept** term), so that the (the new **convention**)

$$h(x) = \sum_{i=0}^d \theta_i x_i = \theta^T x$$



$$h : \mathcal{X} \mapsto \mathcal{Y}$$

How do we pick, or learn, the parameters θ



- One **reasonable** method seems to be to make $h(x)$ **close to y** , at least for the training examples we have.
- We will define a function that measures, for each value of the θ 's, **how close** the $h(x^{(i)})$'s are to the corresponding $y^{(i)}$'s.
- We define the **cost function** (the ordinary least squares):

$$J(\theta) = \frac{1}{2} \sum_{i=1}^n (h_{\theta}(x^{(i)}) - y^{(i)})^2.$$

- We want to choose θ so as to **minimize** $J(\theta)$.



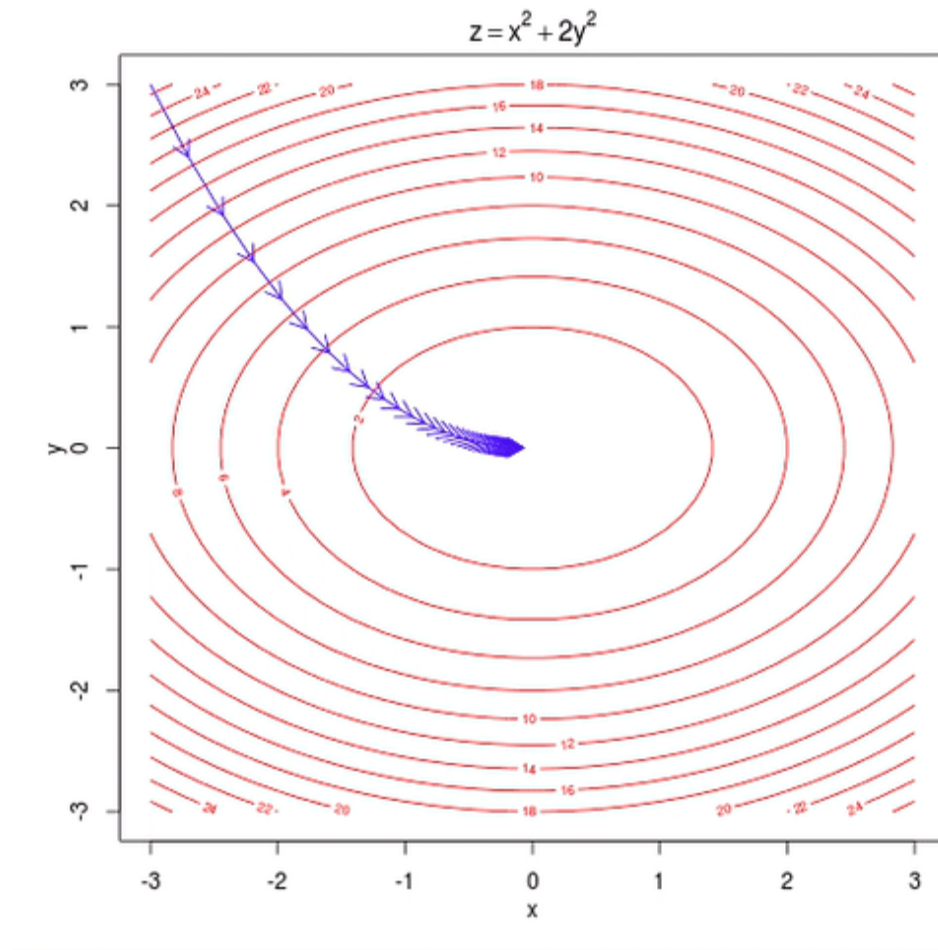
Gradient descent algorithm to find θ

- We update all values of θ_j , $j = 0, \dots, d$

$$\theta_j := \theta_j - \alpha \frac{\partial}{\partial \theta_j} J(\theta)$$

- With some “**initial guess**” for θ , and that **repeatedly** changes θ to make $J(\theta)$ smaller, until **hopefully** we **converge to a value of θ** that minimizes $J(\theta)$.
- α is called the **learning rate**.
- This is a very **natural algorithm** that repeatedly takes a step in the **direction of steepest decrease** of J

Gradient descent



Partial derivative term



- For the case of if we have **only one training example** (x, y)
(neglect the sum in the definition of J)

$$J(\theta) = \frac{1}{2} \sum_{i=1}^n (h_{\theta}(x^{(i)}) - y^{(i)})^2.$$

$$\begin{aligned} \frac{\partial}{\partial \theta_j} J(\theta) &= \frac{\partial}{\partial \theta_j} \frac{1}{2} (h_{\theta}(x) - y)^2 \\ &= 2 \cdot \frac{1}{2} (h_{\theta}(x) - y) \cdot \frac{\partial}{\partial \theta_j} (h_{\theta}(x) - y) \\ &= (h_{\theta}(x) - y) \cdot \frac{\partial}{\partial \theta_j} \left(\sum_{i=0}^d \theta_i x_i - y \right) \\ &= (h_{\theta}(x) - y) x_j \end{aligned}$$

- For a single training example, this gives the update rule:

$$\theta_j := \theta_j + \alpha (y^{(i)} - h_{\theta}(x^{(i)})) x_j^{(i)}.$$

least mean squares (**LMS**)
update rule or **Widrow-Hoff** learning rule.

Widrow-Hoff learning rule.



- The magnitude of the update is **proportional** to the **error** term $(y^{(i)} - h(x^{(i)}))$;
- If we are encountering a training example on which our prediction **nearly matches** the actual value of $y^{(i)}$, then we find that **there is little need to change the parameters**;
- In contrast, a **larger change** to the parameters will be made if our prediction $h(x^{(i)})$ has a **large error** (i.e., if it is very far from $y^{(i)}$).

Modifying method for a training set of more than one example (Solution I)

- **Batch** gradient descent
 - Looks at every example in the **entire training set** on every step, and is called.

Repeat until convergence {

$$\theta_j := \theta_j + \alpha \sum_{i=1}^n (y^{(i)} - h_{\theta}(x^{(i)})) x_j^{(i)}, (\text{for every } j)$$

}

- **Vector notation:**

$$\theta := \theta + \alpha \sum_{i=1}^n (y^{(i)} - h_{\theta}(x^{(i)})) x^{(i)}$$



Solution II: stochastic gradient descent

- To update the parameters according to the **gradient of the error** with respect to that **single training example only**.

Loop {

for $i = 1$ to n , {

$$\theta_j := \theta_j + \alpha (y^{(i)} - h_{\theta}(x^{(i)})) x_j^{(i)}, \quad (\text{for every } j)$$

}

}

$$\theta := \theta + \alpha (y^{(i)} - h_{\theta}(x^{(i)})) x^{(i)}$$

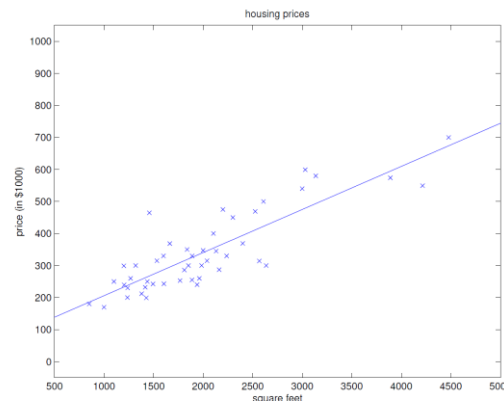
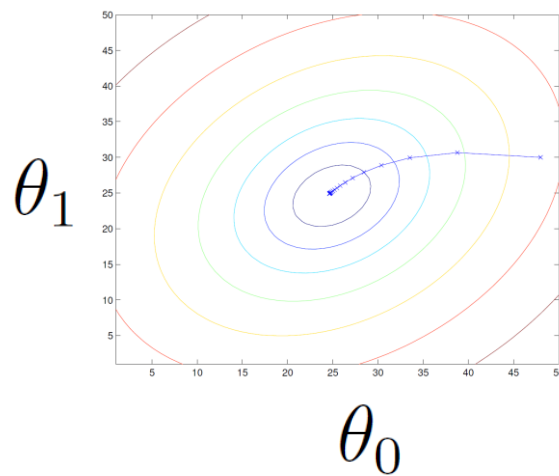
$$\theta := \theta + \alpha \sum_{i=1}^n (y^{(i)} - h_{\theta}(x^{(i)})) x^{(i)}$$

- Often, stochastic gradient descent gets θ “close” to the minimum **much faster** than batch gradient descent.
- When training **set is large**, stochastic gradient descent is **often preferred** over batch gradient descent

Exmample



- J is a **convex quadratic** function.
- The ellipses shown above are the **contours** of a quadratic function.
 $\theta_0 = 71.27, \theta_1 = 0.1345$.



- Also shown is the **trajectory taken** by gradient descent, which was **initialized at (48,30)**.

The normal equations



- Performing the minimization explicitly and **without resorting to an iterative algorithm**.
- Define the **design matrix** X to be the n -by- d matrix that contains the training examples' input values in its rows
- Also, let $\vec{y}$ be the n -dimensional vector containing all the target values from the training set

$$X = \begin{bmatrix} \text{---} & (x^{(1)})^T & \text{---} \\ \text{---} & (x^{(2)})^T & \text{---} \\ & \vdots & \\ \text{---} & (x^{(n)})^T & \text{---} \end{bmatrix} \quad \vec{y} = \begin{bmatrix} y^{(1)} \\ y^{(2)} \\ \vdots \\ y^{(n)} \end{bmatrix}$$

The normal equations



since $h_{\theta}(x^{(i)}) = (x^{(i)})^T \theta$,

$$X\theta - \vec{y} = \begin{bmatrix} (x^{(1)})^T \theta \\ \vdots \\ (x^{(n)})^T \theta \end{bmatrix} - \begin{bmatrix} y^{(1)} \\ \vdots \\ y^{(n)} \end{bmatrix} = \begin{bmatrix} h_{\theta}(x^{(1)}) - y^{(1)} \\ \vdots \\ h_{\theta}(x^{(n)}) - y^{(n)} \end{bmatrix}$$

$$\frac{1}{2}(X\theta - \vec{y})^T (X\theta - \vec{y}) = \frac{1}{2} \sum_{i=1}^n (h_{\theta}(x^{(i)}) - y^{(i)})^2 = J(\theta)$$

To minimize J , let's find its derivatives with respect to θ .



$$\begin{aligned}\nabla_{\theta} J(\theta) &= \nabla_{\theta} \frac{1}{2} (X\theta - \vec{y})^T (X\theta - \vec{y}) \\&= \frac{1}{2} \nabla_{\theta} ((X\theta)^T X\theta - (X\theta)^T \vec{y} - \vec{y}^T (X\theta) + \vec{y}^T \vec{y}) \\a^T b &= b^T a \\&= \frac{1}{2} \nabla_{\theta} (\theta^T (X^T X) \theta - \vec{y}^T (X\theta) - \vec{y}^T (X\theta)) \\&= \frac{1}{2} \nabla_{\theta} (\theta^T (X^T X) \theta - 2(X^T \vec{y})^T \theta) \\\nabla_x b^T x &= b \\&= \frac{1}{2} (2X^T X\theta - 2X^T \vec{y}) \\\nabla_x x^T A x &= 2Ax \\&= X^T X\theta - X^T \vec{y}\end{aligned}$$

The normal equations



- We set its **derivatives to zero**, and obtain the normal equations:

$$X^T X \theta = X^T \vec{y}$$

- The value of θ that minimizes $J(\theta)$ is given in **closed form** by the equation

$$X \theta = \vec{y}$$

$$\theta = (X^T X)^{-1} X^T \vec{y}$$

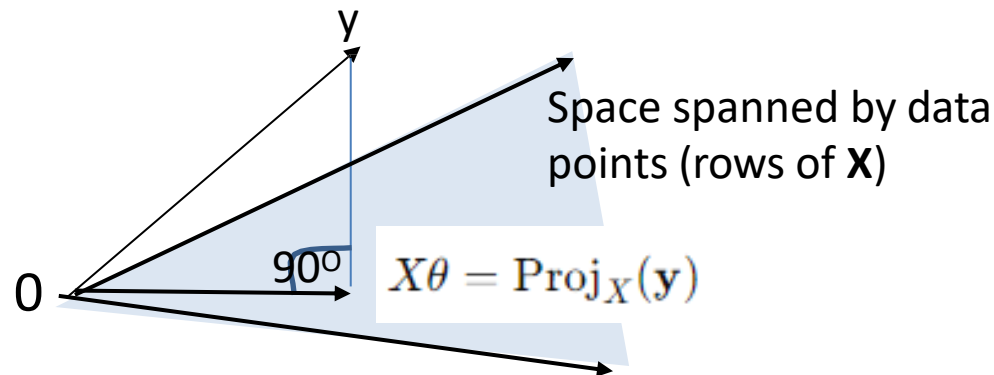
Geometric Interpretation



$$\theta = (X^T X)^{-1} X^T \vec{y}$$

$$X\theta = \text{Proj}_X(\vec{y})$$

$X\theta$ is the projection of the response vector $\vec{y}$ onto the column space of X



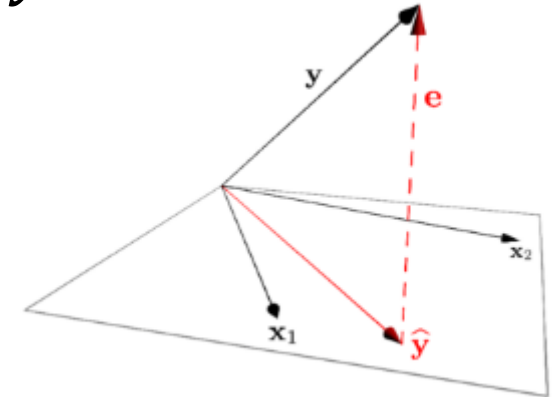
Geometric Interpretation

The prediction $\hat{y}$ is the **orthogonal projection** of y onto the column space of X .

$$\hat{y} = X\theta = X(X^T X)^{-1}X^T y$$

Where:

- $\hat{y}$ is the predicted value (projection),
- X is the design matrix,
- y is the target vector.



The projection $\hat{y}$ minimizes the distance between the actual and predicted target values.

Residual $r = y - \hat{y}$ is orthogonal to the subspace spanned by the columns of X .

Probabilistic interpretation



- **Why** linear regression, and the least-squares cost function J , be a **reasonable choice**?
- Consider following **statistical model**

$$y^{(i)} = \theta^T x^{(i)} + \epsilon^{(i)}$$

- Where $\epsilon^{(i)}$ is an **error term** that captures either **unmodeled effects** or **random noise**.
- $\epsilon^{(i)}$ are **distributed IID** (independently and identically distributed) according to a Gaussian distribution

$$p(\epsilon^{(i)}) = \frac{1}{\sqrt{2\pi}\sigma} \exp\left(-\frac{(\epsilon^{(i)})^2}{2\sigma^2}\right).$$

$$p(y^{(i)}|x^{(i)}; \theta) = \frac{1}{\sqrt{2\pi}\sigma} \exp\left(-\frac{(y^{(i)} - \theta^T x^{(i)})^2}{2\sigma^2}\right)$$

Maximum likelihood



- The likelihood function:

$$L(\theta) = L(\theta; X, \vec{y}) = p(\vec{y}|X; \theta)$$

$$\begin{aligned} L(\theta) &= \prod_{i=1}^n p(y^{(i)} | x^{(i)}; \theta) \\ &= \prod_{i=1}^n \frac{1}{\sqrt{2\pi}\sigma} \exp\left(-\frac{(y^{(i)} - \theta^T x^{(i)})^2}{2\sigma^2}\right) \end{aligned}$$

- The principal of maximum likelihood says that we should **choose θ** so as to **make the data as high probability as possible**. I.e., we should choose θ to maximize $L(\theta)$

Maximize the log likelihood

$$\begin{aligned}\ell(\theta) &= \log L(\theta) \\ &= \log \prod_{i=1}^n \frac{1}{\sqrt{2\pi}\sigma} \exp \left(-\frac{(y^{(i)} - \theta^T x^{(i)})^2}{2\sigma^2} \right) \\ &= \sum_{i=1}^n \log \frac{1}{\sqrt{2\pi}\sigma} \exp \left(-\frac{(y^{(i)} - \theta^T x^{(i)})^2}{2\sigma^2} \right) \\ &= n \log \frac{1}{\sqrt{2\pi}\sigma} - \frac{1}{\sigma^2} \cdot \frac{1}{2} \sum_{i=1}^n (y^{(i)} - \theta^T x^{(i)})^2\end{aligned}$$

- Maximizing $\ell(\theta)$ gives the same answer as **minimizing**

$$\frac{1}{2} \sum_{i=1}^n (y^{(i)} - \theta^T x^{(i)})^2,$$

- Our final choice of θ **did not depend** on what was σ^2

Locally weighted linear regression

- Original linear regression

1. Fit θ to minimize $\sum_i (y^{(i)} - \theta^T x^{(i)})^2$.
2. Output $\theta^T x$.

- **Locally weighted linear regression algorithm**

1. Fit θ to minimize $\sum_i w^{(i)} (y^{(i)} - \theta^T x^{(i)})^2$
2. Output $\theta^T x$.

- $w^{(i)}$'s are **non-negative** valued **weights**.

- If $w^{(i)}$ is small, then the $(y^{(i)} - \theta^T x^{(i)})^2$ error term will be pretty much ignored in the fit.

- A **fairly standard** choice for the weights is

$$w^{(i)} = \exp \left(-\frac{(x^{(i)} - x)^2}{2\tau^2} \right)$$

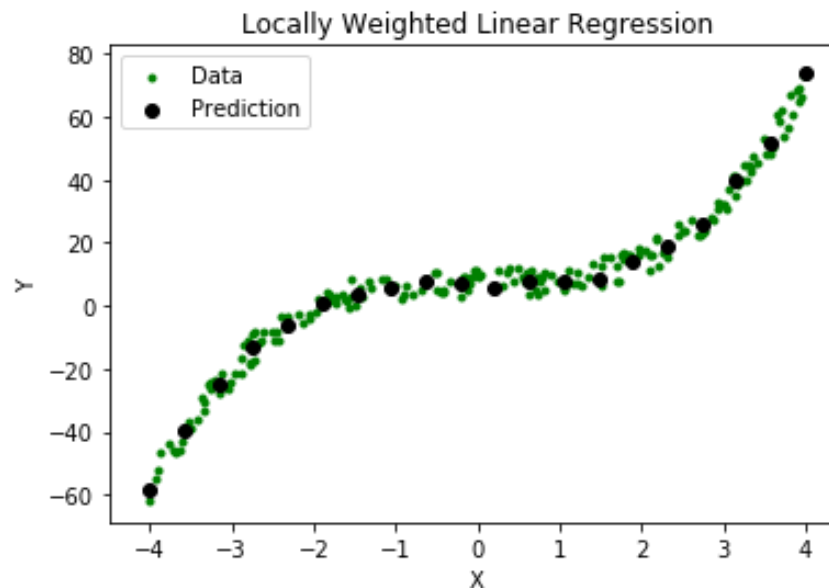
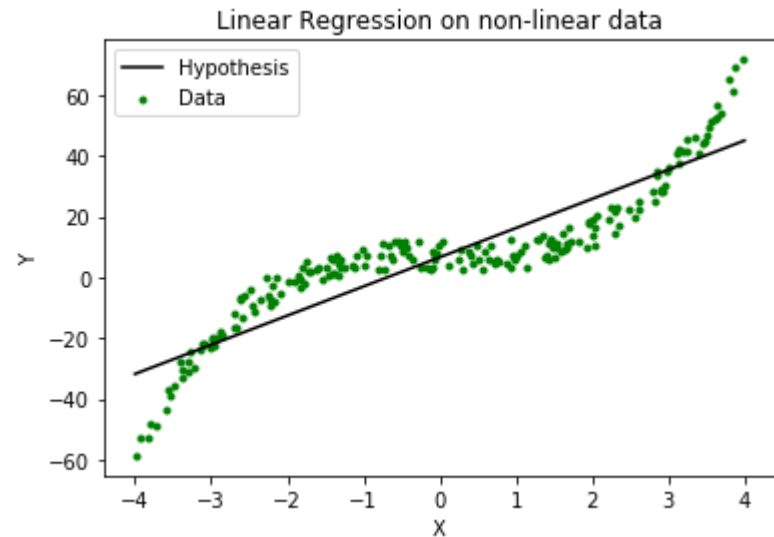
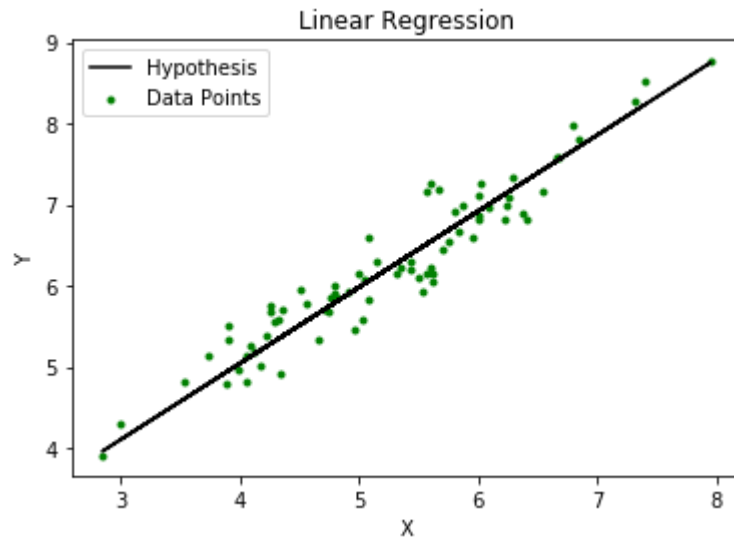
Non-parametric method

$$w^{(i)} = \exp\left(-\frac{(x^{(i)} - x)^2}{2\tau^2}\right)$$



- Note that the weights **depend on the particular point** x at which we're trying to evaluate x .
- θ is chosen giving a much higher “weight” to the (errors on) training examples **close to the query** point x .
- x could be the **position of the center of the peak** in the Bell-shaped function for defining the weights.
- Note that the weights depend on the particular point x at which we're trying to evaluate x .
- τ is called the **bandwidth parameter**. If x is vector-valued, τ is matrix S .
- It is **non-parametric** algorithm:
$$w^{(i)} = \exp(-(x^{(i)} - x)^T \Sigma^{-1} (x^{(i)} - x) / 2)$$
- We need **to keep the entire training set** around. The model does not learn a fixed set of parameters as is done in ordinary linear regression
- **Parameters q are computed individually** for each query point

Example; non-linear relationship between X and Y





Regularized Linear Regression

(adapted from CMU ML course, Aarti Singh)

The normal equations



- We set its **derivatives to zero**, and obtain the normal equations:

$$X^T X \theta = X^T \vec{y}$$

- The value of θ that minimizes $J(\theta)$ is given in **closed form** by the equation

$$\theta = (X^T X)^{-1} X^T \vec{y}$$

Note : $(X^T X)$ is invertible

Q1 : What if $(X^T X)$ is invertible but expensive (d very large)?

Q2 : When is $(X^T X)$ invertible ?

Note: X is $n \times d$ where n is the number of observations and d is the number of features.

Gradient Descent

Even when $(X^T X)$ is invertible, might be computationally expensive if $\mathbf{X}$ is huge.

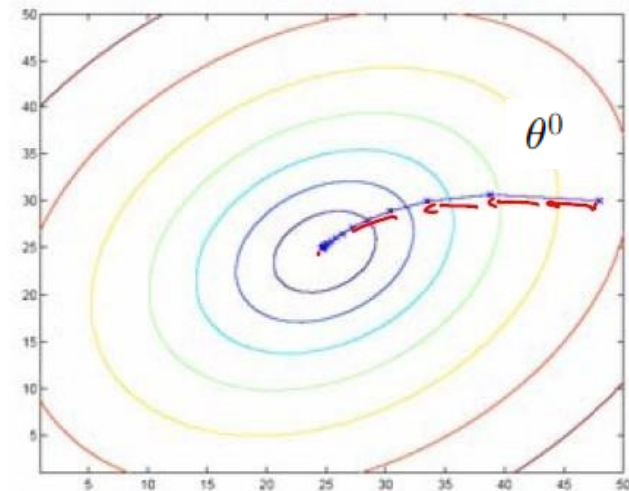
$$\hat{\theta} = \arg \min_{\theta} \frac{1}{n} (X\theta - Y)^T (X\theta - Y) = \arg \min_{\theta} J(\theta)$$

Since $J(\theta)$ is convex, move along negative of gradient

Initialize: θ^0

Update: $\theta^{t+1} = \theta^t - \overset{\text{step size}}{\alpha} \left(\frac{\partial J(\theta)}{\partial \theta} \right)_t$

$$= \theta^t - \alpha \cdot X^T (X\theta^t - Y)$$



Stop: when some criterion met e.g. fixed # iterations, or $\left| \frac{\partial J(\theta)}{\partial \theta} \right|_{\theta_t} < \epsilon$



Recall

Singular value decomposition (SVD)

Singular Value Decomposition (**SVD**) is a matrix factorization technique that decomposes any $n \times d$ matrix into three distinct matrices— U , D , and V^T —each having unique properties that facilitate data analysis and dimensionality reduction

Equation: $X = UDV^T$

- X is the original matrix
- U is an $n \times n$ orthogonal matrix,
- D is an $n \times d$ diagonal matrix with non-negative real numbers on the diagonal,
- V^T is a $d \times d$ orthogonal matrix (the transpose of V).



Recall

Singular value decomposition (SVD)

$$X = U D V^T$$

Components of SVD :

- **Left Singular Vectors (U)**: The columns of U are the left singular vectors of X and form an orthonormal basis for the space of X .
- **Singular Values (D)**: The diagonal entries of D are the singular values of X . These values quantify the strength of the dimensions in X and are sorted in descending order.
- **Right Singular Vectors (V^T)**: The rows V^T (or columns of V) are the right singular vectors of X and form an orthonormal basis for the features space of X .

Applications of SVD:

- Dimensionality Reduction
- Image Compression
- Noise Reduction

Goal: Decompose Matrix X (size $n \times d$)

Given matrix X , we decompose it into three matrices:

$$X = UDV^T$$

- U (size $n \times n$): Orthogonal matrix, columns are eigenvectors of XX^T .
- D (size $n \times d$): Diagonal matrix, containing singular values (square roots of eigenvalues).
- V (size $d \times d$): Orthogonal matrix, columns are eigenvectors of $X^T X$.

Steps:

1. Compute eigenvalues and eigenvectors of $X^T X$ to form V and D .
2. Compute XX^T for eigenvectors to form U .
3. Combine: $X = UDV^T$.

SVD example



Example: X is a 3×2 Matrix

Given matrix:

$$X = \begin{pmatrix} 1 & 0 \\ 0 & 1 \\ 0 & 0 \end{pmatrix}$$

Step 1: Compute $X^T X$:

$$X^T X = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}$$

Eigenvectors form V , and eigenvalues give the diagonal D .

SVD example



Step 2: Compute XX^T :

$$XX^T = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \end{pmatrix}$$

Eigenvectors form U .

Final Decomposition:

$$X = UDV^T$$

With:

$$U = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix}, \quad D = \begin{pmatrix} 1 & 0 \\ 0 & 1 \\ 0 & 0 \end{pmatrix}, \quad V^T = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}$$

Under-Determined System

$$\underbrace{(X^T X)}_{d \times d} \underbrace{\theta}_{d \times 1} = X^T \underbrace{\vec{y}}_{d \times 1}$$

When is $(X^T X)$ invertible ?

Recall: Full rank matrices are invertible. What is rank of $(X^T X)$?

- **Invertible Condition:** The matrix $(X^T X)$ is invertible if it is of full rank. The rank of $(X^T X)$ is equal to the rank of X , which depends on the linear independence of its columns.
- **Full Rank Condition:** A matrix is full rank if all its columns are linearly independent, which means $d \leq n$ and the columns of X span the entire d -dimensional space.

Under-Determined System



For the regression: $y = X\theta + \epsilon$

We want to find θ that minimizes the cost function:

$$\min_{\theta} \|y - X\theta\|^2$$

1. Decompose X using SVD:

$$X = UDV^T$$

2. Substitute into the regression equation:

$$y = UDV^T\theta + \epsilon$$

3. Multiply both sides by U^T :

$$U^Ty = DV^T\theta$$

4. Solve for θ using the pseudoinverse:

$$\theta = VD^+U^Ty$$

SVD makes solving for θ efficient, even for ill-conditioned X .

Under-Determined System: : If $d > n$ (more features than observations), the system is under-determined, which typically means there are infinitely many solutions or no solution without additional constraints.

D^+ refers to the **pseudoinverse** of the diagonal matrix



How is D^+ Computed?

If D is a diagonal matrix of singular values:

$$D = \text{diag}(\sigma_1, \sigma_2, \dots, \sigma_r)$$

Then D^+ is formed by taking the reciprocal of each non-zero singular value:

$$D^+ = \text{diag}\left(\frac{1}{\sigma_1}, \frac{1}{\sigma_2}, \dots, \frac{1}{\sigma_r}\right)$$

For any zero singular values, their reciprocals are also set to zero to avoid division by zero.

Why Use D^+ in Regression?

In the SVD-based solution for regression:

$$\theta = VD^+U^T y$$

The pseudoinverse D^+ allows you to solve for θ even when X is not full rank (i.e., some singular values are zero or close to zero). It handles cases where direct inversion is not possible, ensuring a solution in over- or under-determined systems.



Regularized Least Squares

Ridge Regression (l2 penalty)

Ridge Regression, also known as Tikhonov regularization, modifies linear regression by adding a squared magnitude of coefficient penalty to the loss function.

$$\begin{aligned}\hat{\theta} &= \underset{\theta}{\operatorname{argmin}} \left[\sum_{i=1}^n (y^{(i)} - \theta^T x^{(i)})^2 + \lambda \|\theta\|^2 \right] \\ &= \underset{\theta}{\operatorname{argmin}} [(X\theta - \vec{y})^T (X\theta - \vec{y}) + \lambda \|\theta\|^2]\end{aligned}$$

normal equation adjusted for Ridge Regression : $\hat{\theta} = \underbrace{(X^T X + \lambda I)^{-1}}_{\text{invertible}} X^T Y$

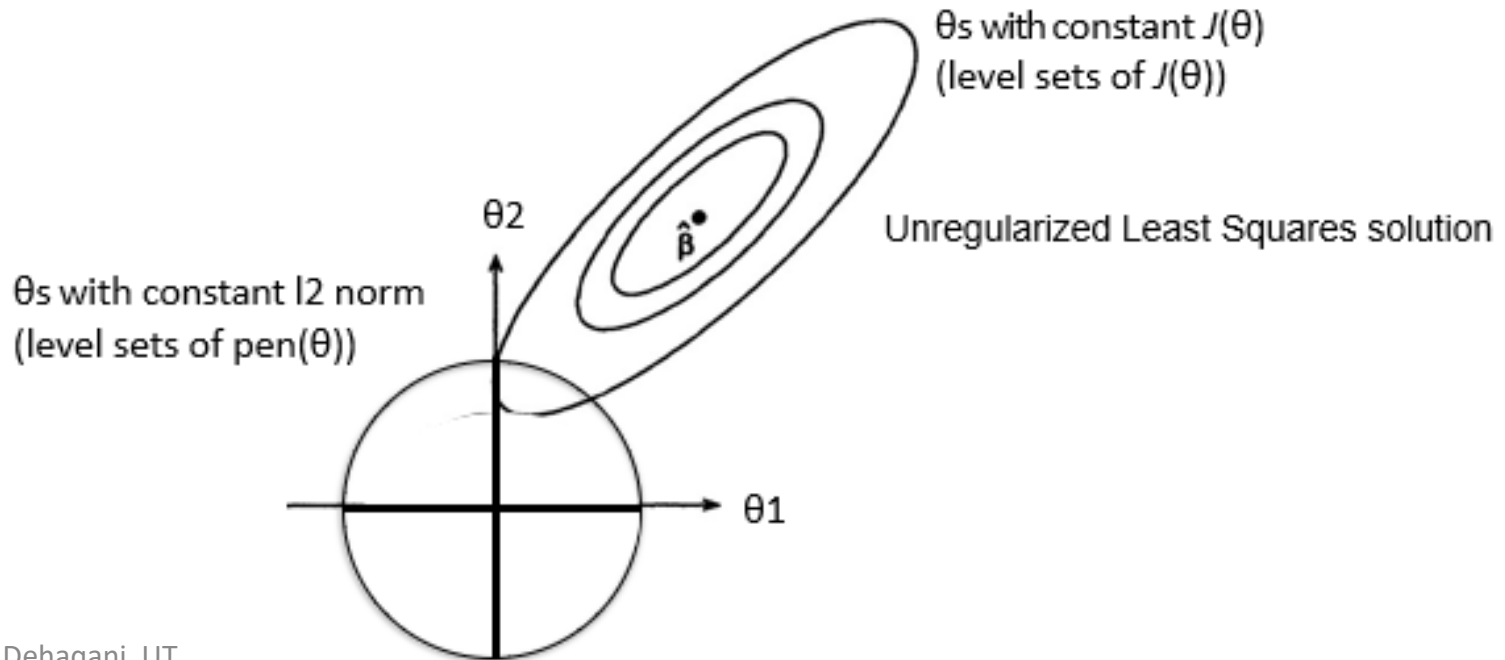
The L2 penalty shrinks the coefficients towards zero, but not exactly zero. This helps in reducing model complexity and preventing overfitting.

Ridge Regression (l2 penalty)



$$\min_{\theta} [(X\theta - \vec{y})^T (X\theta - \vec{y}) + \lambda \|\theta\|^2] = \min_{\theta} [J(\theta) + \lambda \text{pen}(\theta)]$$

Ridge Regression Specifics (L2 norm of θ): $\text{pen}(\theta) = \|\theta\|^2$





Regularized Least Squares

Lasso (l1 penalty)

Lasso Regression, which stands for Least Absolute Shrinkage and Selection Operator, is a type of linear regression that uses an L1 penalty to regularize the coefficients

$$\begin{aligned}\hat{\theta} &= \underset{\theta}{\operatorname{argmin}} \left[\sum_{i=1}^n (y^{(i)} - x^{(i)}\theta)^2 + \lambda \|\theta\|_1 \right] \\ &= \underset{\theta}{\operatorname{argmin}} [(X\theta - \vec{y})^T (X\theta - \vec{y}) + \lambda \|\theta\|_1]\end{aligned}$$

Lasso Regression Specifics:

$$\operatorname{pen}(\theta) = \|\theta\|_1$$

Ridge Regression vs Lasso



L2 Norm of θ (Euclidean Norm) :

$$\|\theta\|_2 = \sqrt{\sum_{i=1}^n \theta_i^2}$$

L1 Norm of θ (Manhattan Norm):

$$\|\theta\|_1 = \sum_{i=1}^n |\theta_i|$$

Both norms are used to regularize regression models, but they influence the models differently:

- **L2 norm (Ridge Regression):** Minimizes the Euclidean length of the coefficient vector, which generally shrinks all coefficients toward zero but does not set them exactly to zero.
- **L1 norm (Lasso Regression):** Promotes sparsity, effectively performing variable selection by setting some coefficients to zero.

Ridge Regression vs Lasso

$$\min_{\theta} [(X\theta - \vec{y})^T (X\theta - \vec{y}) + \lambda \text{pen}(\theta)] = \min_{\theta} J(\theta) + \lambda \text{pen}(\theta)$$

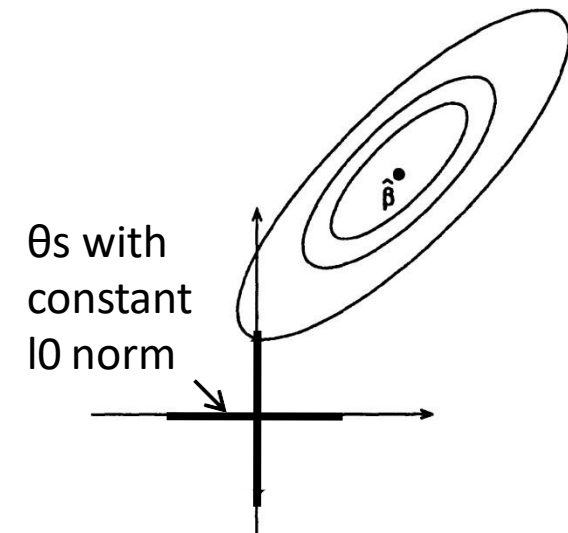
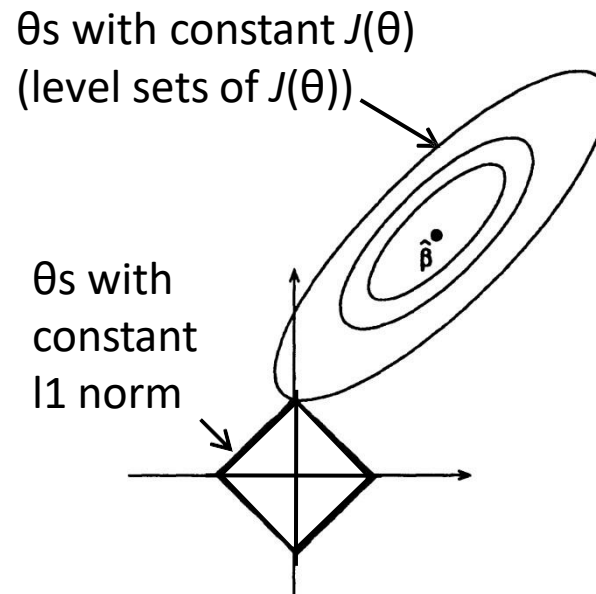
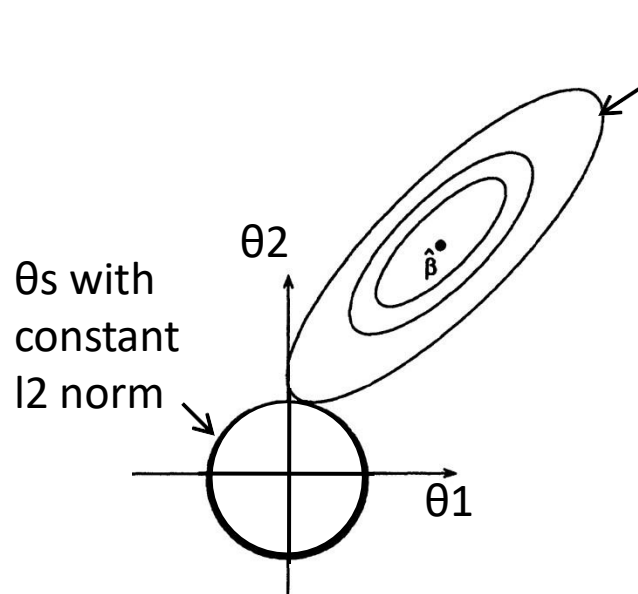
Ridge Regression:

$$\text{pen}(\theta) = \|\theta\|^2$$

Lasso:

$$\text{pen}(\theta) = \|\theta\|_1$$

Ideally l0 penalty,
but optimization
becomes non-convex



Lasso (l1 penalty) results in sparse solutions – vector with more zero coordinates
Good for high-dimensional problems (feature selection) – don't have to store all coordinates, interpretable solution!



Classification and logistic regression

Classification and logistic regression



- Let's now talk about the classification problem.
 - This is just like the regression problem, except that the **values y** we now want to predict take on only a **small number** of **discrete values**.
- For now, we focus on the **binary classification** problem in which y can take on only two values, 0 and 1.
 - In most cases the binary classifier will also **generalize to the multiple-class** case
- $y^{(i)}$ is called the **label for the training** example.
- **Logistic regression:**
 - We could **approach the classification** problem **ignoring** the fact that y is discrete-valued

Logistic function or the sigmoid function.

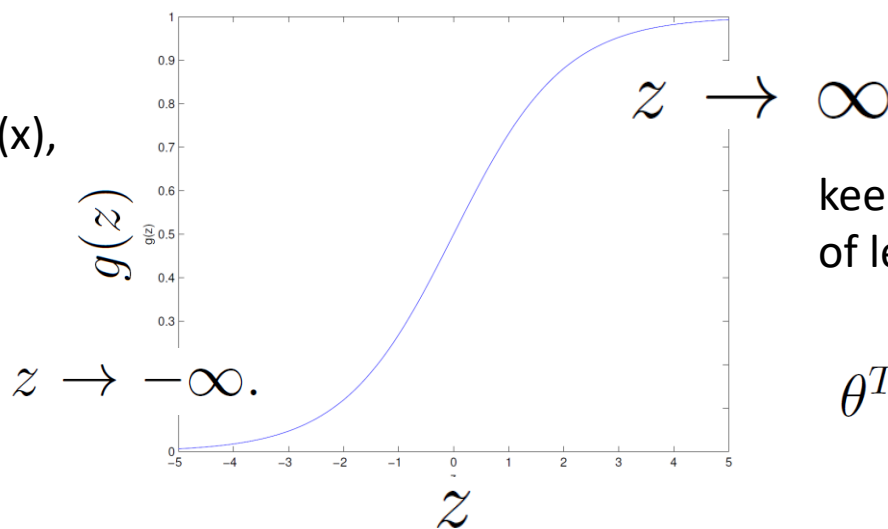


- It also doesn't make sense for $h_q(x)$ to take values larger than 1 or smaller than 0 when we know that $y \in \{0,1\}$;
- We will choose:

$$h_{\theta}(x) = g(\theta^T x) = \frac{1}{1 + e^{-\theta^T x}}$$

$$g(z) = \frac{1}{1 + e^{-z}}$$

$g(z)$, and hence also $h(x)$, is always **bounded** between 0 and 1.



keeping the **convention** of letting $x_0 = 1$, so that:

$$\theta^T x = \theta_0 + \sum_{j=1}^d \theta_j x_j$$



Useful property of the derivative of the sigmoid function,

$$\begin{aligned} g'(z) &= \frac{d}{dz} \frac{1}{1 + e^{-z}} \\ &= \frac{1}{(1 + e^{-z})^2} (e^{-z}) \\ &= \frac{1}{(1 + e^{-z})} \cdot \left(1 - \frac{1}{(1 + e^{-z})} \right) \\ &= g(z)(1 - g(z)) \end{aligned}$$

- **Other functions** that smoothly increase from 0 to 1 can also be used
- The choice of the logistic function is a **fairly natural** one: (GLMs, and generative learning algorithms)

Fitting θ for logistic regression?



- Setting of **probabilistic assumptions**, and then fit the parameters via **maximum likelihood**.

$$P(y = 1 \mid x; \theta) = h_{\theta}(x) \quad P(y = 0 \mid x; \theta) = 1 - h_{\theta}(x)$$

- This can be written more **compactly** as

$$p(y \mid x; \theta) = (h_{\theta}(x))^y (1 - h_{\theta}(x))^{1-y}$$

- **IID assumption** on training examples, then write down the **likelihood of the parameters** as

$$\begin{aligned} L(\theta) &= p(\vec{y} \mid X; \theta) \\ &= \prod_{i=1}^n p(y^{(i)} \mid x^{(i)}; \theta) \\ &= \prod_{i=1}^n (h_{\theta}(x^{(i)}))^{y^{(i)}} (1 - h_{\theta}(x^{(i)}))^{1-y^{(i)}} \end{aligned}$$

Maximizing the log likelihood



$$\begin{aligned}\ell(\theta) &= \log L(\theta) \\ &= \sum_{i=1}^n y^{(i)} \log h(x^{(i)}) + (1 - y^{(i)}) \log(1 - h(x^{(i)}))\end{aligned}$$

- We can **use gradient ascent** (Written in vectorial notation)

$$\theta := \theta + \alpha \nabla_{\theta} \ell(\theta)$$

- Start by working with just **one training** example (x, y) , and take derivatives

$$\begin{aligned}\frac{\partial}{\partial \theta_j} \ell(\theta) &= \left(y \frac{1}{g(\theta^T x)} - (1 - y) \frac{1}{1 - g(\theta^T x)} \right) \frac{\partial}{\partial \theta_j} g(\theta^T x) \\ g'(z) = g(z)(1 - g(z)) &= \left(y \frac{1}{g(\theta^T x)} - (1 - y) \frac{1}{1 - g(\theta^T x)} \right) g(\theta^T x)(1 - g(\theta^T x)) \frac{\partial}{\partial \theta_j} \theta^T x \\ &= (y(1 - g(\theta^T x)) - (1 - y)g(\theta^T x)) x_j \\ &= (y - h_{\theta}(x)) x_j\end{aligned}$$

Stochastic gradient ascent rule and perceptron learning

$$\theta_j := \theta_j + \alpha (y^{(i)} - h_{\theta}(x^{(i)})) x_j^{(i)}$$

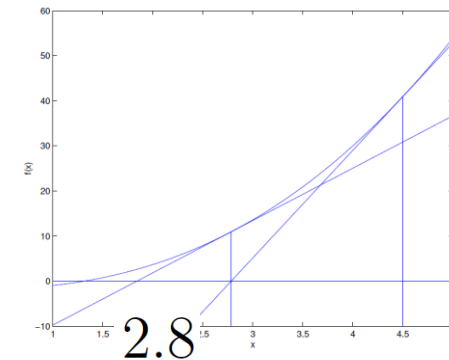
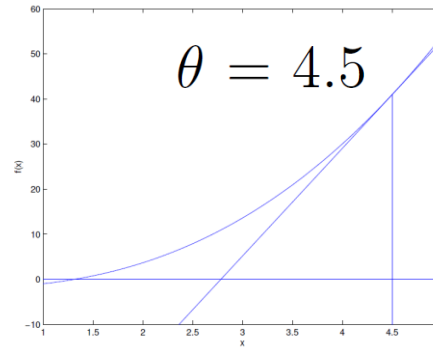
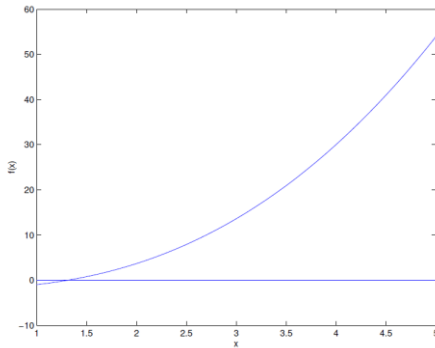
- It is **similar to LMS** update rule; but this is **not** the same algorithm, because **$h_{\theta}(x^{(i)})$ is now defined as a non-linear function of $\theta^T x^{(i)}$**
- There is a **deeper reason** on ending up with the same update rule for a rather different algorithm and learning problem. (GLM models)
- Digression: The **perceptron learning** algorithm
 - Modifying the logistic regression method to “**force**” it to output values that are either 0 or 1 or exactly.

$$g(z) = \begin{cases} 1 & \text{if } z \geq 0 \\ 0 & \text{if } z < 0 \end{cases}$$
 - Using this modified definition of g , and if we use **the same update rule**, then we have the **perceptron learning algorithm**.

Newton's algorithm for maximizing $\ell(\theta)$



- **Newton's method** to find a value of θ so that $f(\theta) = 0$.
- Approximating the function f via a **linear function** that is tangent to f at the current guess $\theta := \theta - \frac{f(\theta)}{f'(\theta)}$



- The maxima of ℓ correspond to points where its first derivative **$\ell'(\theta)$ is zero**. So, by letting $f(\theta) = \ell'(\theta)$, we can use the same algorithm to maximize ℓ . The update rule is: $\theta := \theta - \frac{\ell'(\theta)}{\ell''(\theta)}$

Newton-Raphson method

- **Vector valued** method in **multidimensional space**

$$\theta := \theta - H^{-1} \nabla_{\theta} \ell(\theta)$$

- $\nabla_{\theta} \ell(\theta)$ is, as usual, the **vector of partial derivatives** of $\ell(\theta)$ with respect to the θ_i 's;
- H is an d -by- d matrix (actually, $d+1$ -by- $d+1$, assuming that we include the intercept term) called the **Hessian**,

$$H_{ij} = \frac{\partial^2 \ell(\theta)}{\partial \theta_i \partial \theta_j}.$$

- It is **faster than gradient descent**; however, it is **computationally expensive**. Since it requires finding and **inverting an d -by- d Hessian**
- **Fisher scoring**: Newton's method is applied to maximize the logistic regression